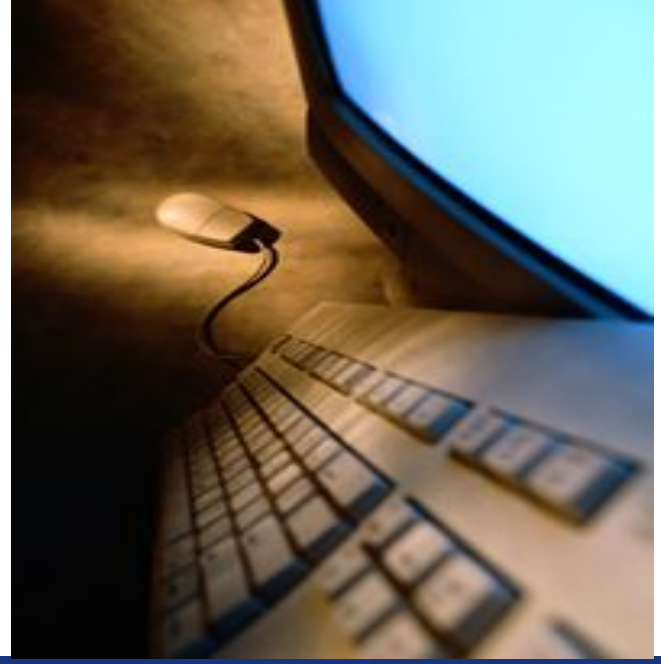
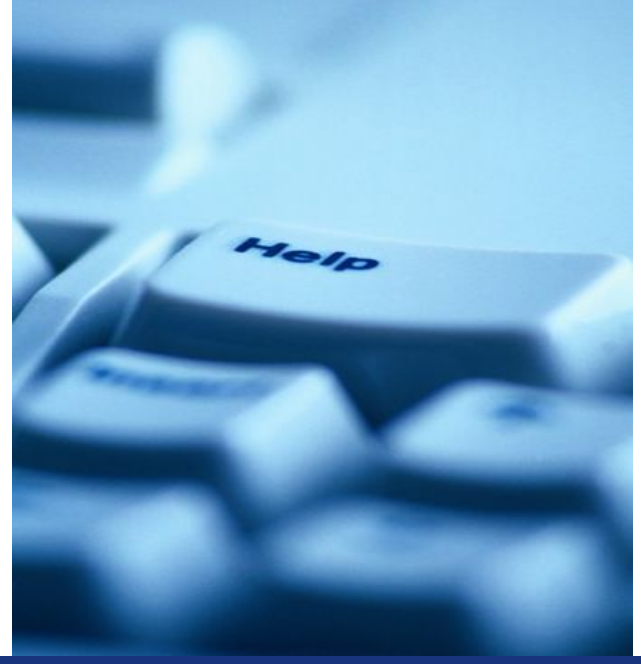
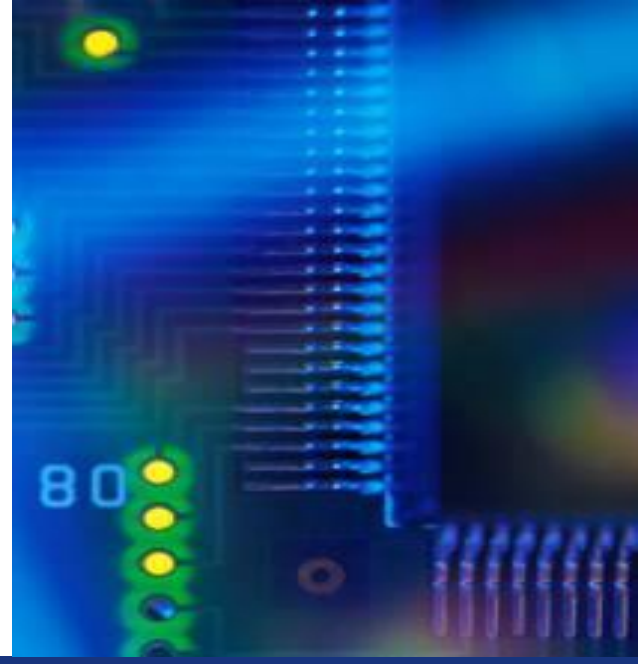
UNIVERSITAS
BUDI LUHUR







FAKULTAS TEKNOLOGI INFORMASI

Pemrograman Java Web dan Enterprise

[PG191/ 3 SKS]

Semester Ganjil 2026/2027

Dosen : Riskiana Wulan S.Kom, M.Kom






Pertemuan 2

Project Maven Pertama & OOP Lanjutan (Collection Framework, Generic & Custom Exception)






Tujuan Pembelajaran

- **Setelah mengikuti perkuliahan ini, mahasiswa mampu:**
  - Membuat dan mengkonfigurasi struktur project Java standar menggunakan Apache Maven.
  - Mengelola data dinamis di memori menggunakan Java Collection Framework (`List` dan `Map`).
  - Menerapkan *Generic* sederhana pada class/interface Java.
  - Menerapkan penanganan error terstruktur menggunakan Custom Exception (`try-catch`, `throw`).
- **Prerequisite:**
  - Mahasiswa telah memahami syntax dasar Java (variabel, tipe data, method) dan telah mencoba program `SimpleCatalog.java` di Minggu ke-1.

Hands On Lab

Step 1: Verifikasi Environment di Terminal

Buka Terminal / Command Prompt dan jalankan perintah berikut:

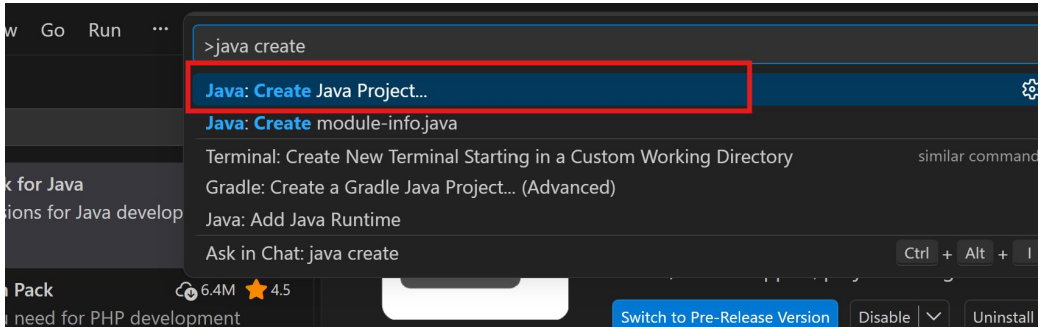
java -version

mvn -version

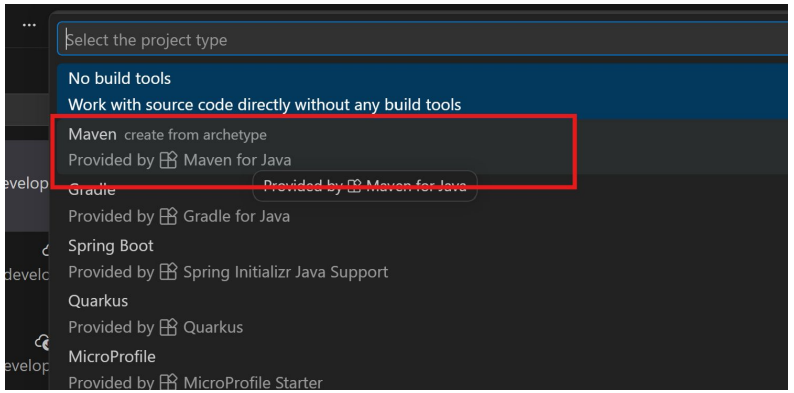
Expected Result: Menampilkan versi Java (minimal versi 21) dan versi Apache Maven yang aktif.

Step 2: Membuat Project Maven Baru di IDE

1. Open Command Palette Menu (Ctrl+Shift+P) -> Ketik keyword Java create lalu pilih. **Java: Create Java Project**

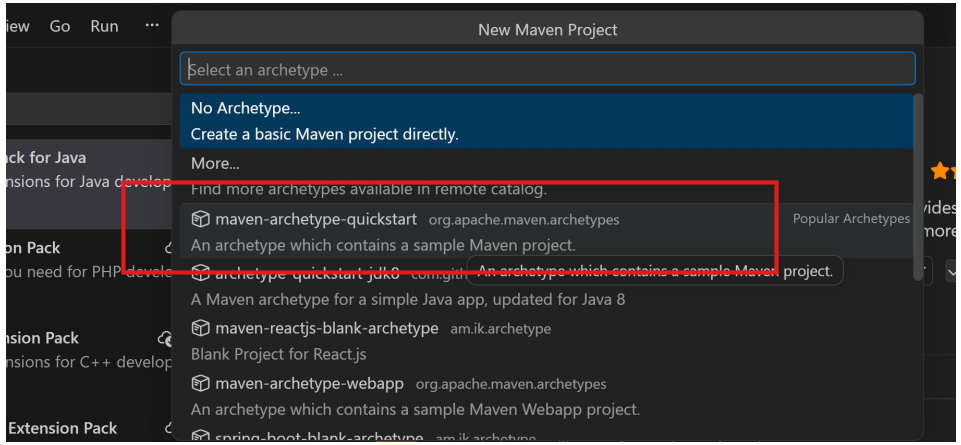


2. pilih Maven

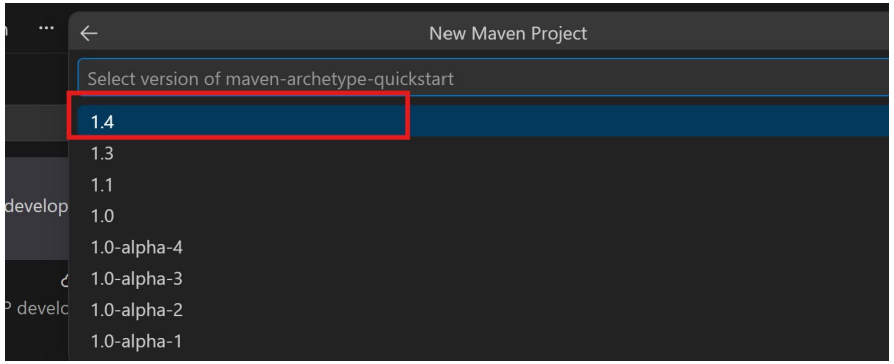


Hands On Lab

3. Maven archetype quickstart



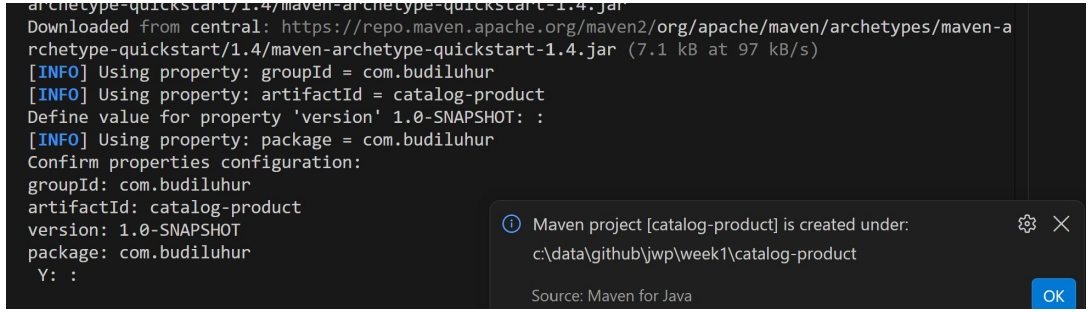
pilih 1.4



4. Isikan identitas project:

- GroupId: com.budiluhur
- ArtifactId: catalog-product

5. cek terminal , lalu enter sampai project maven berhasil tercreate



6.: Membuat Class Utama

- Lokasi File: src/main/java/com/budiluhur/App.java
- Kode Program:






Hands On Lab

```
package com.budiluhur;

public class Application {

    public static void main(String[] args) {

System.out.println("=====================================
======");
        System.out.println("   SISTEM KATALOG PRODUK ENTERPRISE v1.0 ");

System.out.println("=====================================
======");

        // Deklarasi variabel produk
        String productName = "Mouse Wireless Silent";
        double unitPrice = 175000.0;
        int stockQuantity = 12;

        // Memanggil method untuk menampilkan informasi
        printProductDetails(productName, unitPrice, stockQuantity);

// Memeriksa status ketersediaan
        boolean isAvailable = checkStockStatus(stockQuantity);
        System.out.println("Status Ketersediaan: " + (isAvailable ?
"Tersedia" : "Stok Habis"));
    }

    public static void printProductDetails(String name, double price, int
stock) {
        System.out.println("Nama Produk : " + name);
        System.out.println("Harga Satuan: Rp" + price);
        System.out.println("Jumlah Stok : " + stock + " unit");
    }

    public static boolean checkStockStatus(int stock) {
        return stock > 0;
    }
}
```






Hands On Lab

Step 4: Eksekusi Program

Klik kanan pada file `Application.java` -> Pilih Run 'Application.main()'.

Expected Result Output:

```
==========================================
======
   SISTEM KATALOG PRODUK ENTERPRISE v1.0
==========================================
======
Nama Produk : Mouse Wireless Silent
Harga Satuan: Rp175000.0
Jumlah Stok : 12 unit
Status Ketersediaan: Tersedia
```






Mini Challenge

Modifikasi kode `Application.java`:

1. Buat method baru `public static double calculateTotalPrice(int quantity, double price)` yang mengembalikan total harga.
2. Di dalam method `main`, panggil method tersebut untuk menghitung total biaya jika pelanggan membeli 4 unit Mouse keyboard Mechanical.
3. Tampilkan hasil perhitungan total harga tersebut di terminal console.






Apache Maven

- **Apa itu Apache Maven?**

  Maven adalah *build automation & dependency management tool* standar industri untuk pemrograman Java.

  https://maven.apache.org/

- **Mengapa Kita Membutuhkan Maven?**
  1. Manajemen Library Otomatis: Kita cukup mendaftarkan nama library di file pom.xml, Maven akan mengunduhnya secara otomatis.
  2. Struktur Project Terstandar: Setiap developer Java di seluruh dunia menggunakan struktur folder yang sama.

Download:

https://maven.apache.org/download.cgi

Install:

https://maven.apache.org/install.html

Apache Maven™






Apache Maven

- **Struktur Project Standar Maven**

```
product-catalog/
├── pom.xml                          <-- File Konfigurasi Dependency
└── src/
    ├── main/
    │   ├── java/
    │   │   └── com/budiluhur/       <-- Tempat Source Code Java (.java)
    │   └── resources/               <-- Tempat File Konfigurasi / Assets
    └── test/                        <-- Tempat File Unit Testing
```

- **Penting**:
  - Seluruh file `.java` wajib diletakkan di dalam folder `src/main/java/` mengikuti hirarki package.






Apache Maven

- **Struktur Identitas Maven:**
  - `groupId`: Identitas organisasi/paket utama (contoh: `com.budiluhur`).
  - `artifactId`: Nama project/aplikasi (contoh: `catalog-product`).
  - `pom.xml`: File konfigurasi utama (*Project Object Model*).

Berikut ini contoh isi dari pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                            http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.budiluhur</groupId>
    <artifactId>catalog-product</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

</project>
```






Maven Dependency

Maven dependency adalah bagian di pom.xml tempat kita mendaftarkan library eksternal yang dibutuhkan project, tanpa perlu download manual.

- yaitu daftar semua library/framework pihak ketiga yang dipakai project (contoh: JUnit, Spring Boot, MySQL Connector).

- Maven akan otomatis mengunduh library tersebut beserta *dependency*-nya masing-masing dari repository (Maven Central), lalu menyimpannya di local repository **(~/.m2)** agar bisa dipakai berulang tanpa download ulang.

```
(base) PS C:\Users\tellr> cd ~\.m2
(base) PS C:\Users\tellr\.m2> cd repository
(base) PS C:\Users\tellr\.m2\repository> ls


    Directory: C:\Users\tellr\.m2\repository


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         9/15/2026   6:02 AM                antlr
d-----          1/7/2026   1:07 PM                aopalliance
d-----         9/15/2026   6:02 AM                asm
d-----        12/10/2025   7:58 AM                ch
d-----         1/14/2026   6:36 AM                com
d-----        12/10/2025   8:03 AM                commons-codec
d-----         9/15/2026   6:02 AM                commons-collections
d-----        12/10/2025   7:58 AM                commons-io
d-----         9/15/2026   6:02 AM                commons-lang
d-----        12/10/2025   8:44 AM                commons-logging
d-----          1/7/2026   1:32 PM                io
d-----        12/10/2025   7:58 AM                jakarta
d-----          1/7/2026   1:07 PM                javax
d-----         9/15/2026   6:02 AM                jdom
d-----         9/15/2026   6:03 AM                junit
d-----         9/15/2026   6:02 AM                net
d-----         9/15/2026   6:02 AM                org
d-----        12/10/2025   8:44 AM                tools
d-----         2/18/2026  12:26 AM                xmlpull
```






Maven Dependency

Struktur satu <dependency>:

- **groupId** — organisasi pembuat library (contoh: org.junit.jupiter)
- **artifactId** — nama library itu sendiri (contoh: junit-jupiter)
- **version** — versi spesifik yang dipakai
- **scope** — kapan library ini digunakan (compile = default, dipakai di semua tahap; test = hanya saat testing; provided = disediakan environment saat runtime)

```
<dependencies>
    <!-- JUnit 5 untuk unit testing -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>
    </plugins>
</build>
```

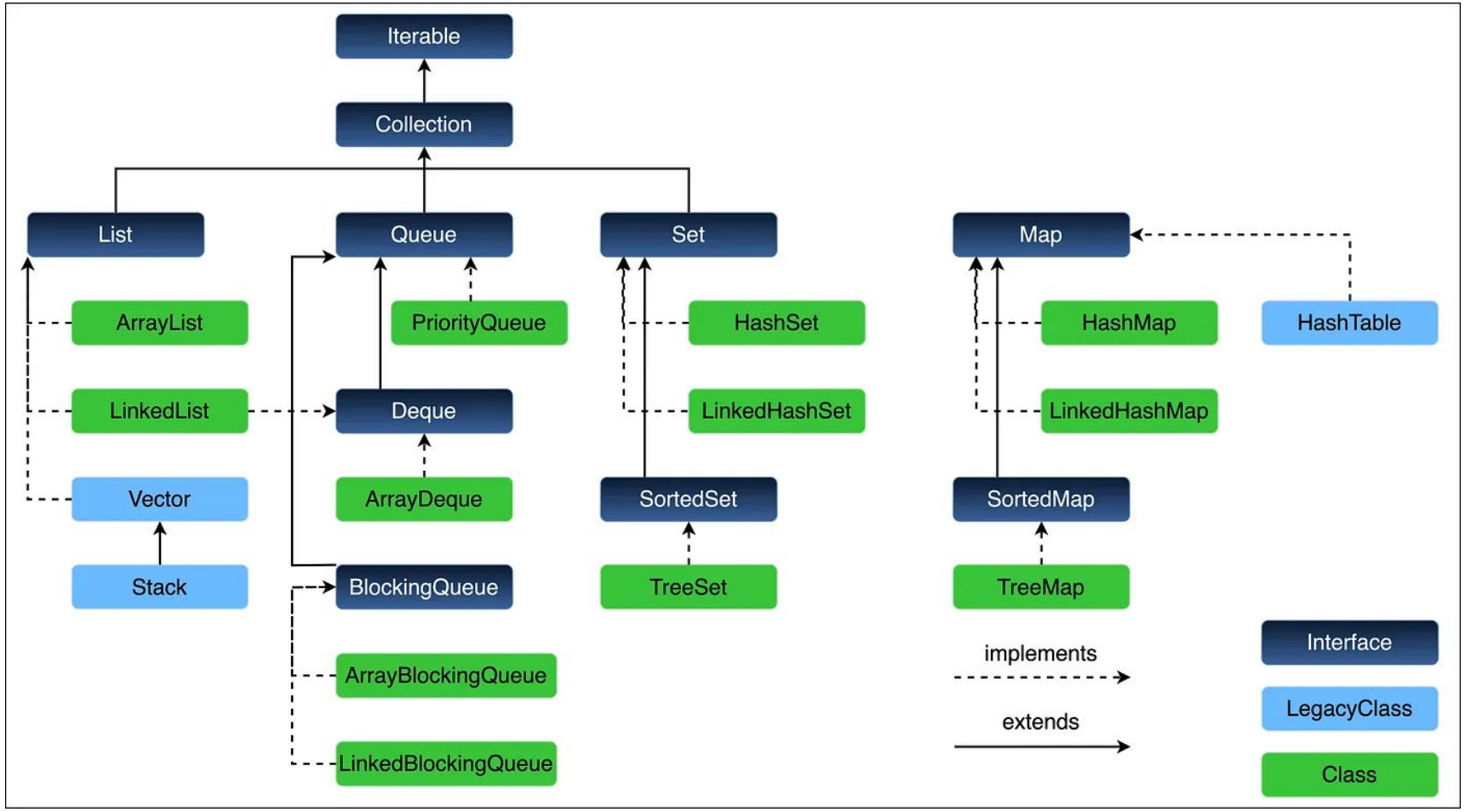






OOP Lanjutan

Java Collection Framework



https://medium.com/@shubhamjha642/mastering-java-collections-in-detail-a-complete-guide-with-code-diagrams-and-use-cases-ec2fc9724c11






Java Collection Framework- List (Array List)

- **List** adalah struktur data berurutan (*ordered collection*) yang ukurannya dapat bertambah atau berkurang secara dinamis.
- Array biasa di Java memiliki ukuran tetap (*fixed size*). List memungkinkan kita menambah data tanpa khawatir kehabisan kapasitas.

Contoh Code Sederhana:

```
List<String> productList = new ArrayList<>();
productList.add("Keyboard Mechanical");
productList.add("Mouse Wireless");


System.out.println("Jumlah produk: " + productList.size());
System.out.println("Produk pertama: " + productList.get(0));
```

- Saat mengembalikan daftar barang dari database, Spring Boot mengemasnya dalam List<Product> sebelum dikonversi menjadi JSON.






Java Collection Framework- Map(HashMap)

- Map adalah struktur data yang menyimpan informasi dalam pasang Key-Value (Kunci - Nilai). Setiap *Key* bersifat unik.

- Cocok untuk pencarian cepat berbasis ID/Kunci tanpa harus melakukan perulangan (*looping*) seluruh data.

Contoh Kode Sederhana:

```
Map<String, Double> priceMap = new HashMap<>();

priceMap.put("PRD-01", 450000.0);

priceMap.put("PRD-02", 175000.0);



double keyboardPrice = priceMap.get("PRD-01");

System.out.println("Harga PRD-01: Rp" +
keyboardPrice);
```






Generic in Java

- *Generic* (disimbolkan dengan `<T>`) adalah fitur Java untuk menentukan tipe data spesifik yang boleh masuk ke dalam sebuah kelas atau collection.
- Digunakan untuk mencegah kesalahan tipe data (*type safety*) pada saat kompilasi (*compile-time*) daripada saat aplikasi berjalan (*runtime*).

- **Perbandingan Kode:**

```
// ❌ Tanpa Generic (Bisa dimasukkan data apa saja, rawan error)


List oldList = new ArrayList();
oldList.add("Keyboard");
oldList.add(1000); // Campur aduk String dan Integer
```

```
// ✅ Dengan Generic (Tipe data terkunci khusus objek Product)


List<Product> productList = new ArrayList<>();
productList.add(new Product("PRD-01", "Keyboard", 450000.0));
// productList.add("String"); // ❌ Compile Error! IDE langsung memberi peringatan
```






Custom Exception Handling

- Merupakan Mekanisme untuk membuat Class menjadi error buatan sendiri yang merepresentasikan masalah bisnis spesifik di aplikasi kita.
- Mengapa Diperlukan? Exception bawaan Java seperti `NullPointerException` terlalu umum. Custom Exception membuat pesan error lebih informatif dan profesional.

- Contoh Pembuatan Custom Exception:

```
package com.budiluhur.catalog.exception;


public class ProductNotFoundException extends Exception
{
    public ProductNotFoundException(String message) {
        super(message);
    }
}
```

- Hubungan dengan Java Web: Saat produk tidak ditemukan di REST API, exception ini dilempar dan ditangkap untuk mengembalikan HTTP Status `404 Not Found`.






Hands On Lab - Refactoring Product Catalog

- Tujuan Lab:
  1. Merefactor project Maven sebelumnya `product-catalog`.
  2. Membuat class Model `Product`.
  3. Membuat class Custom Exception `ProductNotFoundException`.
  4. Membuat class `ProductRepository` menggunakan `List<Product>` (In-Memory CRUD).
  5. Menjalankan aplikasi di `Main.java` menggunakan blok `try-catch`.






Step 1: Struktur Object Product

1: Buat class Product.java

```
package com.budiluhur.catalog.model;

public class Product {
    private String id;
    private String name;
    private double price;

    public Product(String id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Getter & Setter
    public String getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
 @Override
    public String toString() {
        return "[" + id + "] " + name + " - Rp" + price;
    }
}
```






Step 2: Pengelolaan Data InMemory via Repository

2. Buat class ProductRepository.java

```
package com.budiluhur.catalog.repository;

import com.budiluhur.catalog.model.Product;
import com.budiluhur.catalog.exception.ProductNotFoundException;
import java.util.*;

public class ProductRepository {
    private List<Product> productList = new ArrayList<>();

    public void addProduct(Product product) {
        productList.add(product);
    }

    public List<Product> findAll() {
        return productList;
    }

    public Product findById(String id) throws ProductNotFoundException {
        for (Product p : productList) {
            if (p.getId().equalsIgnoreCase(id)) {
                return p;
            }
        }
        throw new ProductNotFoundException("Produk dengan ID '" + id + "' tidak ditemukan!");
    }
}
```






Step 3. Main Application & Testing

3. Ubah App.java sebagai berikut

```
package com.budiluhur.catalog;

import com.budiluhur.catalog.model.Product;
import com.budiluhur.catalog.repository.ProductRepository;
import com.budiluhur.catalog.exception.ProductNotFoundException;

public class Application {
    public static void main(String[] args) {
        ProductRepository repo = new ProductRepository();

        repo.addProduct(new Product("PRD-01", "Keyboard Mechanical",
450000.0));
        repo.addProduct(new Product("PRD-02", "Mouse Wireless",
175000.0));

        System.out.println("=== DAFTAR SELURUH PRODUK ===");
        repo.findAll().forEach(System.out::println);

        System.out.println("\n=== PENCARIAN PRODUK ===");
        try {
            Product p = repo.findById("PRD-99"); // ID tidak ditemukan
            System.out.println("Ditemukan: " + p);
        } catch (ProductNotFoundException e) {
            System.err.println("Error Terjadi: " + e.getMessage());
        }
    }
}
```






Mini Challenge

- Instruksi: Modifikasi `ProductRepository.java` dan `App.java` dari Hands-on Lab:
  1. Buat method `public boolean deleteById(String id) throws ProductNotFoundException` pada `ProductRepository`.
  2. Jika ID ditemukan, hapus produk dari `productList` menggunakan `productList.remove(...)` dan kembalikan nilai `true`.
  3. Jika ID tidak ditemukan, lemparkan `ProductNotFoundException`.
  4. Panggil method tersebut di dalam `main()` menggunakan blok `try-catch`.



SELESAI